Mortal 训练流程全解

离线强化学习麻将 AI 的公式细节与迭代机制

基于仓库源码（version 4）• `train.py` / `train_grp.py` / `model.py` / `reward_calculator.py` / `dataloader.py`

2026 年 9 月

目录

1 训练流水线总览 **2**

2 问题建模与记号 **2**

3 阶段 1：GRP（Game Result Predictor） **3**

3.1 动机与任务定义 . . . 3

3.2 模型与训练目标 . . . 3

3.3 从排列概率到名次矩阵：`calc_matrix` . . . 4

4 阶段 2 前置：奖励构造（全流程的核心推导） **4**

4.1 逐局期望顺位分 . . . 4

4.2 奖励 = 期望顺位分的差分 . . . 4

4.3 训练样本的六元组 . . . 5

5 网络结构（version 4） **5**

5.1 Brain：ResNet1d + 通道注意力 . . . 5

5.2 DQN 头：dueling 结构（合法动作均值） . . . 6

5.3 AuxNet . . . 6

6 总损失：三项逐一推导 **6**

6.1 第一项：蒙特卡洛 Q 回归 . . . 6

6.2 第二项：CQL 保守正则（仅离线） . . . 7

6.3 第三项：名次辅助任务 . . . 7

7 优化器与学习率 **7**

7.1 分组 weight decay 的 AdamW . . . 7

7.2 学习率调度 . . . 8

7.3 其余工程细节 . . . 8

8 两层迭代机制 **8**

8.1 内层：梯度下降 8

8.2 外层：test_play 评估与 baseline 自我提升 8

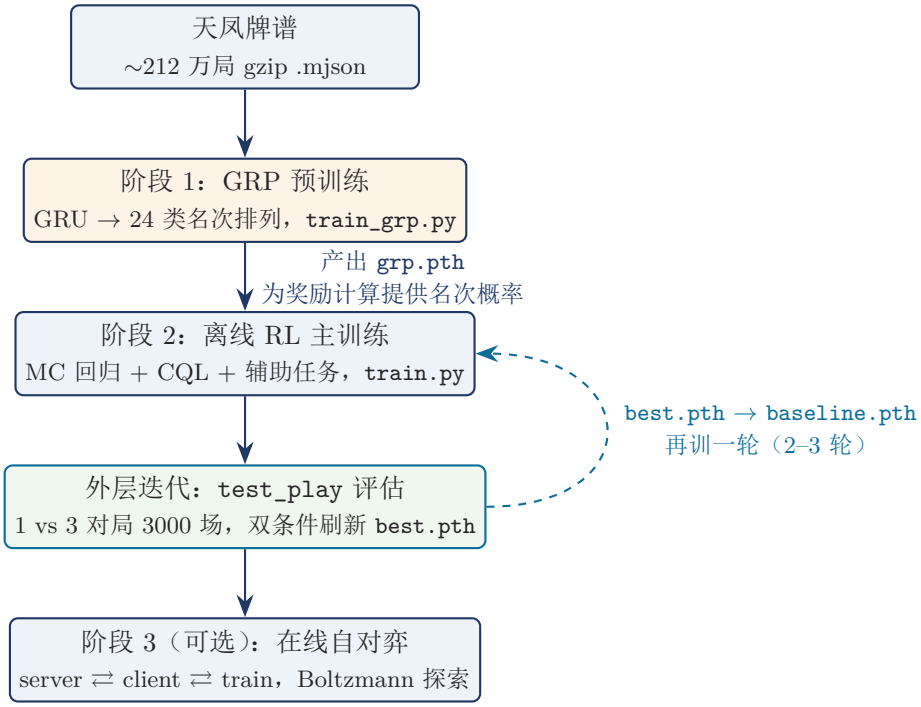
9 阶段 3（可选）：在线自对弈 **9**

10 超参数速查表（本机 config.toml） **9**

11 记忆锚点 **9**

1 训练流水线总览

Mortal 的训练分为三个阶段，其中主训练内部还有**两层迭代**：内层是梯度下降更新网络参数；外层是 “打败 baseline → 自己成为新 baseline → 再打败更强的自己” 的自我提升循环。



本文按数据流向逐步展开：先讲 GRP 如何训练（§3），再讲它如何生成奖励（§4），然后是网络结构（§5）、总损失（§6）、优化细节（§7），最后是两层迭代机制（§8）与在线模式（§9）。

2 问题建模与记号

麻雀的层级结构：一场**半庄**（game）⊃ 若干**局**（kyoku，东 1 . . . 南 4）⊃ 若干**决策**（step）。约定：

- 观测 $o_i \in \mathbb{R}^{1012\times 34}$：34 个牌种位置上的 1012 通道手工特征平面（手牌、副露、舍牌历史、立直信息、场况等）；

- 动作 $a_i \in \{0, \dots, 45\}$：46 个动作（34 种切牌 + 立直/杠/碰/吃/拔北/跳过等）；
- 合法动作掩码 $m_i \in \{0,1\}^{46}$；
- 顺位分向量 $\boldsymbol{p} = [6, 4, 2, 0]^\top$（`config.toml` 的 `pts`）；
- 折扣因子 $\gamma = 1$（本机配置），即纯蒙特卡洛设定。

MDP 视角下，Mortal 学习的是动作价值函数 $Q(o, a)$，目标是“当前决策对最终成绩（以期望顺位分计）的贡献”。由于终局信号只有最终名次一个，极度稀疏，因此整个训练的第一步是**用 GRP 把终局信号稠密化到每一局**。

3 阶段 1：GRP（Game Result Predictor）

3.1 动机与任务定义

动机：如果奖励只在半庄结束时给出一次，信用分配（credit assignment）将无法进行。GRP 的任务是：**给定每局结束时的场况与四家分数，预测最终的完整名次排列**。有了逐局的名次概率，就能构造逐局的中间奖励。

输入：一场半庄是逐局序列，第 k 局的特征为 7 维向量

$$x_k = \big[\ \underbrace{g_k}_{\text{grand_kyoku}},\ \underbrace{h_k}_{\text{本场景}},\ \underbrace{r_k}_{\text{立直棒}},\ s_{k,0},\ s_{k,1},\ s_{k,2},\ s_{k,3}\ \big] \in \mathbb{R}^7,$$

其中 g_k 把东 1 … 西 4 编码为 $0 \dots 11$，四家分数除以 10^4 归一（开局每人 2.5）。

输出：24 维 logits，对应 $4! = 24$ 种名次排列 π 的打分。

3.2 模型与训练目标

网络为两层 GRU（hidden size 64），取最终隐状态过两层 MLP：

$$\boldsymbol{z} = \mathrm{GRU}(x_1, \dots, x_K) \in \mathbb{R}^{128}, \qquad \boldsymbol{\ell} = \mathrm{MLP}(\boldsymbol{z}) \in \mathbb{R}^{24}.$$

标签：真实名次排列在 24 种排列中的下标。`get_label` 的实现是把 24 个候选排列与真实 `rank_by_player` 逐元素比对，取全等的那个下标 $y^\star$。训练目标就是普通交叉熵：

$$\mathcal{L}_{\mathrm{GRP}} = -\log \frac{\exp(\ell_{y^\star})}{\sum_{j=0}^{23} \exp(\ell_j)}, \qquad \text{AdamW},\ \eta = 10^{-5},\ \text{float64}.$$

本机训练结果（23.3 万步）：val loss $3.17 \to 2.47$，val acc $= 22.6\%$。注意这是 **24 类精确命中完整排列**的准确率，随机基线仅 $1/24 \approx 4.2\%$，且 val 与 train 全程贴合——对奖励构造而言这个精度已足够。

3.3 从排列概率到名次矩阵：calc_matrix

softmax 得到 24 个排列的概率后，需要**边缘化**成 “每个玩家拿每个名次” 的 4×4 矩阵 M：

$$\boxed{M_{p,r} \;=\; \Pr(\text{玩家}p \text{ 最终第}r{+}1 \text{ 名}) \;=\; \sum_{\pi:\ \pi(p)=r} \operatorname{softmax}(\boldsymbol{\ell})_\pi}$$

即对所有 “玩家 p 恰好排在第 r 位” 的排列求概率和。代码中用一个预计算的 $(4, 24)$ 索引表 `perms_t` 完成这个分组求和。

注. 为什么要先预测 24 类再边缘化，而不是直接输出 4×4？因为名次排列之间存在强耦合（一人升必有一人降），直接输出 16 维会丢失这种联合约束；24 类分类让 softmax 天然处理互斥与归一。

4 阶段 2 前置：奖励构造（全流程的核心推导）

4.1 逐局期望顺位分

对某玩家 p，GRP 给出**每局结束时**的名次概率向量 $M_{k,p} \in \mathbb{R}^4$（第 k 局结束、r 取 $0 \ldots 3$）。把逐局向量堆叠后，在末尾拼接**真实最终名次的 one-hot 向量** $\boldsymbol{e}_{\text{final}}$：

$$R = \left[\, M_{0,p};\ M_{1,p};\ \ldots;\ M_{K,p};\ \boldsymbol{e}_{\text{final}} \,\right] \in \mathbb{R}^{(K+2)\times 4}.$$

每局结束时的期望顺位分：

$$e_k = R_k^\top \boldsymbol{p}, \qquad \boldsymbol{p} = [6, 4, 2, 0]^\top.$$

4.2 奖励 = 期望顺位分的差分

第 k 局的奖励定义为相邻两项的差：

$$\boxed{r_k = e_{k+1} - e_k}$$

为什么这样设计：

(1) 打得好（和了跳满）⇒ GRP 上调你的一位概率 ⇒ $e_{k+1} > e_k$，奖励为正；被炸庄则为负。终局信号被平滑地摊到了每一局。

(2) 序列的**终点**是真实名次的 one-hot（e_{K+1} 是真实顺位分），所以整条奖励链锚定在真实结局上，不是模型自说自话。

(3) **伸缩守恒**：$\sum_k r_k = e_{K+1} - e_0$，即逐局奖励之和恰等于 “真实顺位分 − 开局期望顺位分” ——总账始终等于真实成绩。

注 (Sanity check). 开局时（东 1 开始前）四家对称，$e_0 = \frac{1}{4}(6+4+2+0) = 3$；若最终拿 2 位，$\sum_k r_k = 4 - 3 = 1$。奖励总量与实际名次严格对应。

4.3 训练样本的六元组

`dataloader.py` 把每场牌谱逐决策拆成：

$$\Big(\underbrace{o_i}_{\mathbb{R}^{1012\times 34}},\ \underbrace{a_i}_{\text{实际动作}},\ \underbrace{m_i}_{\text{合法掩码}},\ \underbrace{n_i}_{\text{steps_to_done}},\ \underbrace{r_{k(i)}}_{\text{该局奖励}},\ \underbrace{y_i}_{\text{该局末真实名次}}\Big).$$

两个细节：

(a) n_i（steps_to_done）的递推：从场尾倒序扫描，

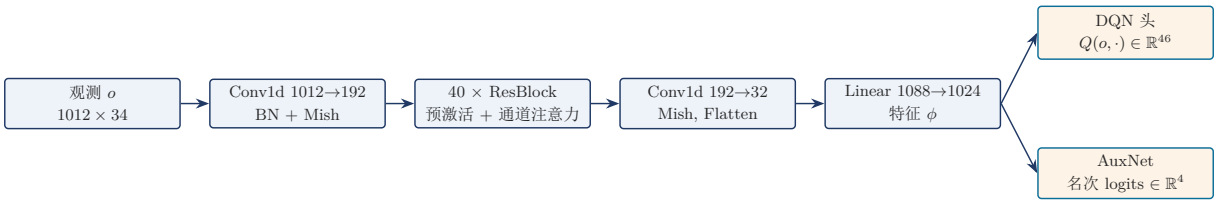
$$n_i = \begin{cases} 0, & \text{第} i \text{ 步后本场已结束,} \\ n_{i+1} + \text{1}\!\!\text{1}[\texttt{apply_gamma}_i], & \text{否则,} \end{cases}$$

即 “从 i 到本场结束还剩几次计入折扣的决策”，作为折扣 γ^{n_i} 的指数。

(b) 辅助标签 y_i：由各局末分数做一次双 `argsort`（分数 → 名次）得到 $y_i = \texttt{rank}[\,k(i)+1\,]$，即 “该动作所在的这局打完后的真实名次”。

工程上每批读 15 个文件（约 1 万条决策）进 buffer、shuffle 后吐出，近似全局随机采样。

5 网络结构（version 4）



5.1 Brain：ResNet1d + 通道注意力

把牌桌看成 34 个位置（牌种）上的特征平面，用 Conv1d 处理。每个 ResBlock 为预激活结构

$$x \mapsto x + \mathrm{CA}\big(\mathrm{Conv}(\mathrm{Mish}(\mathrm{BN}(\mathrm{Conv}(\mathrm{Mish}(\mathrm{BN}(x)))))))\big),$$

并挂一个 **ChannelAttention**（CBAM 式通道门控）：对空间维分别取均值与最大值，过同一个共享 MLP 后相加、sigmoid，得到逐通道权重

$$\boldsymbol{w} = \sigma\Big(\mathrm{MLP}\big(\mathrm{mean}_c(x)\big) + \mathrm{MLP}\big(\mathrm{max}_c(x)\big)\Big), \qquad x' = \boldsymbol{w} \odot x.$$

均值捕获整体活跃度，最大值捕获 “是否存在关键模式”，二者互补；这让网络自己学会哪些特征通道重要。BatchNorm 的 momentum $= 0.01$（慢统计，适配分布漂移）。

5.2 DQN 头：dueling 结构（合法动作均值）

v4 的 DQN 头是单个 Linear $1024 \to 1+46$，拆出标量 V 与优势向量 A：

$$Q(o,a) = V(\phi) + A(\phi,a) - \frac{1}{|\mathcal{A}_{\text{legal}}|}\sum_{a' \in \mathcal{A}_{\text{legal}}} A(\phi,a'), \qquad Q(o,a_{\text{illegal}}) = -\infty$$

与标准 dueling 的两点不同：

(1) 优势函数只对**合法动作**归零均值（掩码求和再除以合法数），非法动作不参与归一；

(2) 非法动作直接置 $-\infty$：argmax 永远落在合法动作上，且后面 CQL 的 logsumexp 自动把 $e^{-\infty}=0$ 的项排除。

5.3 AuxNet

一个无偏置线性层 $\mathbb{R}^{1024} \to \mathbb{R}^4$，从共享特征 ϕ 预测“本局结束后的名次”，迫使 ϕ 编码对局走势信息，防止表示坍缩。

6 总损失：三项逐一推导

$$\mathcal{L} = \underbrace{\tfrac{1}{2}\,\mathrm{MSE}\big(Q(o,a),\ \gamma^n r_k\big)}_{\text{DQN 蒙特卡洛回归}} + \underbrace{5\Big(\log\textstyle\sum_{a'} e^{Q(o,a')} - Q(o,a)\Big)}_{\text{CQL 保守正则}} + \underbrace{0.2\ \mathrm{CE}\big(\mathrm{AuxNet}(\phi),\ y\big)}_{\text{名次辅助任务}}$$

6.1 第一项：蒙特卡洛 Q 回归

目标值 $\tilde{q}_i = \gamma^{n_i}\cdot r_{k(i)}$。本机 $\gamma=1$，于是

$$\tilde{q}_i = r_{k(i)},$$

即**每个动作的 Q 直接回归到它所在那一局造成的期望顺位分变化**。这是关键设计决策：

- **不用 TD bootstrap**：没有 target network、没有 $Q(s')$ 自举，也就没有自举偏差——在离线数据上格外稳；
- **局内信用均摊**：同一局的所有动作共享同一个 r_k（该局打得好，局中每个决策都被记功）；跨局的信用由 GRP 差分天然分离；
- 损失系数 $\frac{1}{2}$ 使梯度恰为残差：$\nabla\frac{1}{2}(Q-\tilde{q})^2 = Q-\tilde{q}$。

6.2 第二项：CQL 保守正则（仅离线）

$$\mathcal{L}_{\text{CQL}} = \mathbb{E}_{o\sim\mathcal{D}}\Big[\underbrace{\log\sum_{a'} e^{Q(o,a')}}_{\text{压低全体 Q}} - \underbrace{Q(o,a)}_{\text{抬数据中的动作}}\Big], \qquad \text{权重5.}$$

动机：离线数据固定，分布外（OOD）动作的 Q 值没有任何监督，回归误差会让它们被**系统性高估**；一旦推理时 argmax 选到这种动作，行为就会崩坏。logsumexp 是全体合法动作 Q 的软最大值，最小化它等价于**压低所有动作的 Q**；减去 $Q(o,a)$ 则**把数据里实际被选择（天凤高手）的动作抬回来**。两项之差恒非负（logsumexp $\geq$ 任意单项），最优解处数据动作与软最大值的差距最小。

注 (Sanity check). 若模型对所有动作输出相同 Q，$\mathcal{L}_{\text{CQL}} = \log|\mathcal{A}_{\text{legal}}| > 0$；只有当数据动作的 Q 显著高于其他动作时才趋近 0。这解释了 tensorboard 中 `cql_loss` 应为正值并逐渐下降。在线模式下数据是自己产的，无 OOD 问题，故此项关闭。

6.3 第三项：名次辅助任务

$$\mathcal{L}_{\text{aux}} = \text{CE}\big(\text{AuxNet}(\phi),\ y\big), \qquad \text{权重0.2.}$$

标签 y 是“本局结束后的真实名次”（§4）。作用类似 Suphx 的辅助预测任务：给表示 ϕ 加一个与最终成绩直接相关的监督，缓解纯 Q 回归的表示退化。

7 优化器与学习率

7.1 分组 weight decay 的 AdamW

参数被分成两组（`train.py` 遍历 `named_modules`）：

$$\theta_{\text{decay}} = \{\text{Linear / Conv1d 的 weight}\}, \qquad \theta_{\text{no-decay}} = \{\text{bias、BatchNorm 参数}\}.$$

AdamW 的**解耦**衰减只施加在第一组：

$$\theta \leftarrow \theta - \eta\big(\hat{m}_t/(\sqrt{\hat{v}_t}+\varepsilon)\big) - \eta\lambda\,\theta, \qquad \lambda = 0.1 \text{ 仅作用于 } \theta_{\text{decay}}.$$

为什么 BN 参数不衰减：BN 的 scale/shift 控制特征归一化强度，对其做 L2 衰减会无意义地把激活压向零，损害表示能力——这是标准实践。

7.2 学习率调度

`LinearWarmUpCosineAnnealingLR`（线性 warmup + 余弦退火）：

$$\eta_t = \begin{cases} \eta_{\text{init}} + \dfrac{\eta_{\text{peak}} - \eta_{\text{init}}}{t_{\text{warm}}}\, t, & t < t_{\text{warm}}, \\ \eta_{\text{final}} + \dfrac{\eta_{\text{peak}} - \eta_{\text{final}}}{2}\Big(1 + \cos\dfrac{(t - t_{\text{warm}})\pi}{t_{\max} - t_{\text{warm}}}\Big), & t_{\text{warm}} \le t < t_{\max}, \\ \eta_{\text{final}}, & t \ge t_{\max}. \end{cases}$$

本机配置 $\eta_{\text{peak}} = \eta_{\text{final}} = 10^{-4}$、$t_{\text{warm}} = 0$，即**恒定学习率** 10^{-4}（离线数据可以反复 epoch，不需要退火收敛）。

7.3 其余工程细节

- **AMP 混合精度**：autocast + GradScaler；损失除以 `opt_step_every` 实现梯度累积，累积满才 `scaler.step` + `update`；
- `scheduler.step()` 每个 batch 调用一次（不是每个优化步）；
- 每 `save_every` = 400 步：保存含 optimizer/scheduler/scaler 全状态的 checkpoint（可无缝续训），并把**预测 Q 与目标 Q 的直方图**写入 tensorboard——两者叠合程度是回归质量的直接可视化；
- $\gamma = 1$ 时目标量级在 ±6 以内，Q 值不会爆炸，这是可以用恒定 lr 长训的前提之一。

8 两层迭代机制

8.1 内层：梯度下降

每个 batch 计算 $\mathcal{L}$（§6），反向传播更新 Brain、DQN、AuxNet 的全部参数。离线模式对整个文件列表跑一个 epoch 即退出（便于人工控制），重启时从 checkpoint 续训。

8.2 外层：test_play 评估与 baseline 自我提升

每 `test_every` = 20000 步触发一次对局评估：

(1) 用 `OneVsThree` 竞技场打 3000 场：**1 个当前模型 vs 3 个 baseline**（`baseline.pth`，本机为社区 298k 步权重），全程 eval、无探索；

(2) 统计 `avg_rank`（2.5 = 随机）、`avg_pt` 及和牌/放铳/立直率等行为指标；

(3) **双条件**刷新最好模型：

$$\texttt{better} \iff (\texttt{avg_pt} \ge \texttt{best.avg_pt}) \text{ 且 } (\texttt{avg_rank} \le \texttt{best.avg_rank}),$$

两个指标同时不劣才复制为 `best.pth`——宁可保守，防止评估噪声把 checkpoint 刷掉；

(4) **自我提升外循环**：指标稳定后执行 `cp best.pth baseline.pth` 再训一轮。评估对手变成更强的自己，模型被迫继续进步，可重复 2–3 轮。

注. 这与 Suphx 的思路同源：监督/离线学到的是 “模仿强者”，而对局评估闭环提供的是 “超越强者” 的压力。双条件判定相当于对噪声评估加了一个保守的接受域。

9 阶段 3（可选）：在线自对弈

三进程架构：`server.py`（参数分发与牌谱中转）⇄ `client.py`（自对弈，可多开）⇄ `train.py`（`online=true`）。与离线模式的差异：

组件	离线模式	在线模式
数据来源	天凤牌谱（固定）	client 自对弈实时产生
探索	无（数据已含人类策略）	Boltzmann：$\varepsilon = 0.005$，$T = 0.05$
CQL 正则	开启（×5）	**关闭**（无 OOD 问题）
BatchNorm	正常更新	**冻结**（`freeze_bn=true`）
参数同步	—	每 `submit_every` 步推给 server

在线时 client 以 Boltzmann 探索采样动作（$\Pr(a) \propto e^{Q/T}$，温度 $T = 0.05$ 使分布接近贪心、保留少量探索），牌谱经 server 缓冲后被训练进程 `drain` 取走。BN 冻结是因为自对弈数据分布随参数快速漂移，小批量统计会抖动。

10 超参数速查表（本机 `config.toml`）

类别	参数	值
数据	batch_size / file_batch_size / num_workers	512 / 15 / 0（Windows 共享内存坑）
环境	γ / pts	1 / $[6, 4, 2, 0]$
网络	conv_channels / num_blocks / version	192 / 40 / 4
损失	CQL 权重 / 辅助权重	5 / 0.2
优化器	AdamW：betas / ε / weight_decay	$(0.9, 0.999)$ / 10^{-8} / 0.1（分组）
调度	peak / final	10^{-4} / 10^{-4}（恒定）
节奏	save_every / test_every / test 局数	400 / 20000 / 3000
GRP	GRU 2×64 / lr / batch	24 类输出 / 10^{-5} / 512

11 记忆锚点

1. **“GRP 把终局信号摊到每一局”**：奖励 = 期望顺位分的差分 $r_k = \Delta\,\mathbb{E}[\text{顺位分}]$，且伸缩守恒 $\sum_k r_k =$ 真实顺位分 − 开局期望。同原理：Suphx 的 global reward prediction；围棋的胜率差分 shaping。

2. **“离线 RL = 回归 + 保守项”**：见过的动作抬高、没见过的压低；在线后把 CQL 关掉就是它的逆向应用。

3. **“两层迭代”**：内层 AdamW 更新参数，外层 `best.pth` → `baseline.pth` 更新对手——评估对手变强，模型才被迫继续变强。